EXP NO: 7 — VERSION 2

GENERATIVE MODELS WITH GANS: CREATING AND TRAINING A GENERATIVE ADVERSARIAL NETWORK

AIM

To construct and train a Generative Adversarial Network (GAN) using TensorFlow/Keras on the MNIST dataset to generate new synthetic images of handwritten digits.

PROCEDURE

1. Import the required TensorFlow, Keras, NumPy and Matplotlib libraries.
2. Load the MNIST dataset and normalize the images to the range −1 to 1.
3. Define the Generator network using dense, batch normalization and transposed convolution layers.
4. Define the Discriminator network using convolution, dropout and dense layers.
5. Define the generator and discriminator loss functions.
6. Create Adam optimizers for both networks.
7. Train the GAN using real MNIST images and generated images.
8. Generate and display synthetic digit images at different training stages.

CODE

```
import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf

from tensorflow.keras import Sequential

from tensorflow.keras.layers import Dense, Reshape,

BatchNormalization from tensorflow.keras.layers import LeakyReLU,

Conv2DTranspose

from tensorflow.keras.layers import Conv2D, Flatten, Dropout

from tensorflow.keras.datasets import mnist
```

```
(x_train, _), (_, _) = mnist.load_data()



x_train = x_train.astype("float32") / 127.5 - 1
x_train = np.expand_dims(x_train, axis=-1)

latent_size = 64
batch_size = 128
epochs = 30

generator = Sequential([
 Dense(7 * 7 * 128, input_shape=(latent_size,)),
 BatchNormalization(),
 LeakyReLU(),
 Reshape((7, 7, 128)),
 Conv2DTranspose(128, 4, strides=2, padding="same"),
BatchNormalization(),
 LeakyReLU(),
 Conv2DTranspose(64, 4, strides=2, padding="same"),
BatchNormalization(),
 LeakyReLU(),
 Conv2DTranspose(1, 3, padding="same", activation="tanh")
])

discriminator = Sequential([
```

```
 Conv2D(64, 4, strides=2, padding="same", input_shape=(28, 28, 1)),
LeakyReLU(),
 Dropout(0.4),
 Conv2D(128, 4, strides=2, padding="same"),
 LeakyReLU(),


 Dropout(0.4),
 Flatten(),
 Dense(1, activation="sigmoid")
])

bce = tf.keras.losses.BinaryCrossentropy()

g_optimizer = tf.keras.optimizers.Adam(
 learning_rate=0.0002,
 beta_1=0.5
)

d_optimizer = tf.keras.optimizers.Adam(
 learning_rate=0.0002,
 beta_1=0.5
)

dataset = tf.data.Dataset.from_tensor_slices(x_train)
dataset = dataset.shuffle(60000).batch(batch_size, drop_remainder=True)

@tf.function
```

```
def train_batch(real_images):
 noise = tf.random.normal((batch_size, latent_size))

 with tf.GradientTape() as generator_tape:
 fake_images = generator(noise, training=True)
 fake_output = discriminator(fake_images, training=True)


 generator_error = bce(tf.ones_like(fake_output), fake_output)

 generator_gradients = generator_tape.gradient(
 generator_error,
 generator.trainable_variables
 )

 g_optimizer.apply_gradients(
 zip(generator_gradients, generator.trainable_variables)  )

 noise = tf.random.normal((batch_size, latent_size))

 with tf.GradientTape() as discriminator_tape:
 fake_images = generator(noise, training=True)

 real_output = discriminator(real_images, training=True)
fake_output = discriminator(fake_images, training=True)

 real_error = bce(tf.ones_like(real_output), real_output)
fake_error = bce(tf.zeros_like(fake_output), fake_output)
```

```
 discriminator_error = real_error + fake_error

 discriminator_gradients = discriminator_tape.gradient(
discriminator_error,
 discriminator.trainable_variables


 )

 d_optimizer.apply_gradients(
 zip(discriminator_gradients, discriminator.trainable_variables)  )

 return generator_error, discriminator_error

fixed_noise = tf.random.normal((16, latent_size))

def show_generated_images(epoch):
 generated = generator(fixed_noise, training=False)
generated = (generated + 1) / 2

 plt.figure(figsize=(6, 6))

 for i in range(16):
 plt.subplot(4, 4, i + 1)
 plt.imshow(generated[i, :, :, 0], cmap="gray")
plt.axis("off")
```

```
 plt.suptitle("GAN Generated Digits - Epoch " + str(epoch))
plt.tight_layout()
 plt.show()

for epoch in range(epochs):
 g_losses = []


 d_losses = []

 for real_batch in dataset:
 g_loss, d_loss = train_batch(real_batch)
g_losses.append(g_loss.numpy())
d_losses.append(d_loss.numpy())

 print(
 "Epoch:",
 epoch + 1,
 "Generator Loss:",
 round(np.mean(g_losses), 4),
"Discriminator Loss:",
 round(np.mean(d_losses), 4)
 )

 if (epoch + 1) % 5 == 0:
 show_generated_images(epoch + 1)
```

show_generated_images(epochs)

OUTPUT







RESULT

Thus, the Generative Adversarial Network was successfully constructed and trained using the MNIST dataset. The Generator learned to create synthetic handwritten digit images, while the Discriminator learned to distinguish between real and generated images. The generated images became more realistic as training progressed.